

Asignatura: Desarrollo de Aplicaciones Móviles Nativas.

Tema: Localización GPS.

INTRODUCCIÓN.

Localización Geográfica.

Una de las funciones exclusivas de las aplicaciones para dispositivos móviles es el reconocimiento de la ubicación. Los usuarios de dispositivos móviles los llevan a todas partes. Con la API de servicios de ubicación de Google Play se simplifica la gestión de la ubicación al combinar diferentes fuentes de datos de ubicación, como GPS, red celular y Wi-Fi, para ofrecer la ubicación más precisa y eficiente.

La API de los servicios de localización de Google Play (Fused Location Provider API) es el método preferido para agregar la función de reconocimiento de la ubicación a una aplicación para Android. Incluye funcionalidades que permitirán hacer lo siguiente:

- Determinar la ubicación del dispositivo.
- Escuchar cambios en la ubicación.
- Determinar el medio de transporte si el dispositivo está en movimiento.
- Crear y supervisar regiones geográficas predefinidas, conocidas como perímetros virtuales.

Las APIs de ubicación permiten compilar fácilmente aplicaciones con capacidad de ahorro de energía y reconocimiento de la ubicación. Al igual que el SDK de Maps para Android, la API de Location se distribuye como parte del SDK de Servicios de Google Play.

En un móvil, la localización usualmente es con el GPS y con la red telefónica. Con el método `getAllProviders()` de la clase `LocationManager` se realiza la localización. Con una instancia del administrador de localización `getSystemService(LOCATION_SERVICE)` se obtiene la lista de proveedores del servicio:

```
LocationManager locationManager = (LocationManager) getSystemService(LOCATION_SERVICE);  
List<String> listaProviders = locationManager.getAllProviders();
```

La clase `LocationManager` posee al método llamado `isProviderEnabled()` al que se le pasa el nombre del proveedor. Para su localización:

```
LocationManager.NETWORK_PROVIDER // para la red de telefonía.  
LocationManager.GPS_PROVIDER    // para el GPS.
```

La clase `LocationManager` posee el método `getLastKnownLocation(String provider)`, que devuelve la última posición que se obtuvo a través del proveedor desde su uso. Se debe suscribir al evento que se lanza cuando un proveedor recibe nuevos datos sobre la localización actual.

La clase `LocationListener` posee métodos asociados a los eventos que se reciben del proveedor:

<code>onLocationChanged(location)</code>	Para actualizar la posición.
<code>onProviderDisabled(provider)</code>	Cuando el proveedor se deshabilita.
<code>onProviderEnabled(provider)</code>	Cuando el proveedor se habilita.
<code>onStatusChanged(provider, status, extras)</code>	Cuando el proveedor cambia su estado. Sus valores pueden estar entre OUT OF SERVICE, TEMPORARILY UNAVAILABLE, AVAILABLE.

Para implantar el método `onLocationChanged`, se instancia, por ejemplo:

```
LocationListener locListener = new LocationListener() {  
    public void onLocationChanged(Location location) {  
        mostrarPosicion(location);  
    }  
    public void onProviderDisabled(String provider) {  
        lblEstado.setText("Proveedor en OFF");  
    }  
    public void onProviderEnabled(String provider) {  
        lblEstado.setText("Proveedor en ON");  
    }  
}
```

```

        public void onStatusChanged(String provider, int status, Bundle extras){
            lblEstado.setText("Status del Proveedor: " + status);
        }
    };

```

Si se necesita acceder a la ubicación del usuario, primero se solicita el permiso correspondiente.

Android ofrece dos permisos de ubicación: **ACCESS_COARSE_LOCATION** y **ACCESS_FINE_LOCATION**. El permiso que se seleccione determinará la exactitud de la ubicación que mostrará la API.

android.permission.ACCESS_COARSE_LOCATION	Permite que la API muestre la ubicación aproximada del dispositivo. El permiso proporciona una estimación de la ubicación del dispositivo a partir de los servicios de ubicación.
android.permission.ACCESS_FINE_LOCATION	Permite que la API determine la ubicación más precisa posible mediante los proveedores de ubicación disponibles, incluido el sistema de posicionamiento global (GPS), así como Wi-Fi y datos móviles.

Lo siguiente es importante para solicitar los permisos en tiempo de ejecución y se puede incluir dentro del **MainActivity.java**, invocando por ejemplo, al método **checkLocationPermission()**, como se indica enseguida:

```

private void checkLocationPermission() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this, new String[]{
            Manifest.permission.ACCESS_FINE_LOCATION}, 123);
    }
}

@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
    if (requestCode == 123) {
        if (grantResults.length > 0 && grantResults[0] ==
            PackageManager.PERMISSION_GRANTED) {
            // Permiso concedido, continuar con la localización
        } else {
            // Permiso denegado, muestra mensaje o desactiva la funcionalidad
        }
    }
}

```

DESARROLLO.

Ejemplo 1.

Crear un nuevo proyecto en Android Studio. Copiar y modificar los archivos correspondientes con los códigos siguientes.

1. Archivo **activity_main.xml**.

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">
    <TextView
        android:id="@+id/tv_location"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="La latitud y longitud se muestran aquí:"
        android:textSize="18sp"
        android:layout_centerInParent="true" />
</RelativeLayout>

```

2. Archivo **MainActivity.java**.

```

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.*;
import android.os.Bundle;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

public class MainActivity extends AppCompatActivity {
    private TextView tvLocation;
    private LocationManager locationManager;
    private LocationListener locationListener;
    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);
        tvLocation = findViewById(R.id.tv_location);
        locationManager = (LocationManager) getSystemService(LOCATION_SERVICE);
        locationListener = new LocationListener() {
            @Override
            public void onLocationChanged(@NonNull Location location) {
                double latitude = location.getLatitude();
                double longitude = location.getLongitude();
                tvLocation.setText("Latitud: " + latitude + "\nLongitud: " + longitude);
            }
        };
        @Override
        public void onStatusChanged(String provider, int status, Bundle extras) {}
        @Override
        public void onProviderEnabled(@NonNull String provider) {}
        @Override
        public void onProviderDisabled(@NonNull String provider) {}
    };
    requestLocationUpdates();
}

private void requestLocationUpdates() {
    if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED
        && ActivityCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_COARSE_LOCATION) !=
        PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 1);
        return;
    }
    locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 0, 0,
        locationListener);
}

@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[]
permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == 1 && grantResults.length > 0 && grantResults[0] ==
        PackageManager.PERMISSION_GRANTED) {
        requestLocationUpdates();
    }
}
}

```

3. En el archivo **AndroidManifest.xml** se asignan los permisos que se indican con letra en **negritas**.

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.gpslocation">

```

```

<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<application
    android:allowBackup="true"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.GPSLocation">
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>

</manifest>

```

Ejecutar la aplicación para visualizar la localización del móvil.

Ejemplo 2.

Antes de iniciar:

- Agregar la dependencia de maps-utils en el archivo build.gradle del módulo app:
implementation 'com.google.maps.android:android-maps-utils:2.2.5'
- Asegurarse de reemplazar LA_CLAVE_DE_API con la clave de API de Google Maps.
- Configurar el SDK de Google Maps en el proyecto de Android Studio. Obtener una clave de API de Google Maps y configurarla en el archivo AndroidManifest.xml.

Crear un nuevo proyecto en Android Studio. Copiar y modificar los archivos correspondientes con los códigos siguientes.

1. Archivo activity_main.xml.

```

<RelativeLayout
    xmlns:android=http://schemas.android.com/apk/res/android
    xmlns:tools=http://schemas.android.com/tools
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <com.google.android.gms.maps.MapView
        android:id="@+id/mapView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</RelativeLayout>

```

2. Archivo MainActivity.java.

```

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import com.google.android.gms.maps.*;
import com.google.android.gms.maps.model.*;
import com.google.maps.android.kml.KmlLayer;
import org.xmlpull.v1.XmlPullParserException;
import java.io.IOException;

public class MainActivity extends AppCompatActivity implements OnMapReadyCallback {
    private MapView mapView;
    private GoogleMap googleMap;
    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);
        mapView = findViewById(R.id.mapView);
    }
}

```

```

        mapView.onCreate(savedInstanceState);
        mapView.getMapAsync(this);
    }
    @Override
    public void onMapReady(GoogleMap map) {
        googleMap = map;
        try {
            // Carga el archivo KML desde la carpeta res/raw
            KmlLayer kmlLayer = new KmlLayer(googleMap, R.raw.location_data,
                getApplicationContext());
            kmlLayer.addLayerToMap();
            // Opcional: Agregar manualmente un marcador
            googleMap.addMarker(new MarkerOptions()
                .position(new LatLng(-34, 151))
                .title("Marcador de ejemplo"));
        } catch (XmlPullParserException | IOException e) {
            e.printStackTrace();
        }
    }
    @Override
    protected void onResume() {
        super.onResume();
        mapView.onResume();
    }
    @Override
    protected void onPause() {
        super.onPause();
        mapView.onPause();
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        mapView.onDestroy();
    }
    @Override
    protected void onLowMemory() {
        super.onLowMemory();
        mapView.onLowMemory();
    }
}

```

3. En el archivo **AndroidManifest.xml** se asignan los permisos que se indican con letra en negritas.

```

<manifest
    xmlns:android=http://schemas.android.com/apk/res/android
    package="com.example.kmllocation">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <application android:allowBackup="true" android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true"
        android:theme="@style/Theme.KMLLocation">
        <meta-data android:name="com.google.android.maps.v2.API_KEY"
            android:value="TU_CLAVE_DE_API"/>
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>

```

Guardar el siguiente archivo con el nombre `location_data.kml` en la carpeta `res/raw` del proyecto. Si la carpeta `raw` no existe, crearla dentro de la carpeta `res`.

4. Archivo `location_data.kml`.

```
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <Placemark>
      <name>Ubicación 1</name>
      <Point>
        <coordinates>-122.0822035425683,37.42228990140251,0</coordinates>
      </Point>
    </Placemark>
    <Placemark>
      <name>Ubicación 2</name>
      <Point>
        <coordinates>-122.085,37.422,0</coordinates>
      </Point>
    </Placemark>
  </Document>
</kml>
```

Ejecutar la aplicación para visualizar la localización del móvil.

NOTA. Generar un reporte de la ejecución de los ejercicios, incluyendo las imágenes obtenidas durante la ejecución de las aplicaciones; guardarlo con el nombre `AlumnoGPSgrupo.doc` y enviarlo al sitio indicado por el profesor.

ANEXO.

Una forma alternativa de probar la localización en un emulador, cuando no se posee acceso a un GPS, es con el uso de archivos con extensión `.kml`, como se describe enseguida.

Para revisar la forma de reducir el tiempo de espera y el comportamiento de la aplicación con mensajes `log`, los cuales se generan cuando:

- El proveedor de localización cambia de estado y se muestra el nuevo estado.
- Se recibe una actualización de la posición y se muestran las nuevas coordenadas recibidas.

Se utiliza la siguiente configuración para simular la localización. El archivo `.kml` se guarda en la carpeta del proyecto. Ahora, se modifica el código como se indica, es decir:

```
private void actualizarPosicion() {
    locationManager = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
    Location location =
    locationManager.getLastKnownLocation(LocationManager.GPS_PROVIDER);
    muestraPosicion(location);
    locationManager = new LocationListener() {
        public void onLocationChanged(Location location) {
            muestraPosicion(location);
        }
        public void onProviderDisabled(String provider) {
            lblEstado.setText("Provider OFF");
        }
        public void onProviderEnabled(String provider) {
            lblEstado.setText("Provider ON");
        }
        public void onStatusChanged(String provider, int status, Bundle extras) {
            Log.i("LocAndroid", "Provider Status: " + status);
            lblEstado.setText("Provider Status: " + status);
        }
    };
};
```

```

locationManager.requestLocationUpdates( locationManager.GPS_PROVIDER, 15000, 0,
locationListener);
}
private void muestraPosicion(Location loc) {
    if(loc != null) {
        lblLatitud.setText("Latitud: " + String.valueOf(loc.getLatitude()));
        lblLongitud.setText("Longitud: " + String.valueOf(loc.getLongitude()));
        lblPrecision.setText("Precision: " + String.valueOf(loc.getAccuracy()));
        Log.i("LocAndroid", String.valueOf(loc.getLatitude() + " - " +
        String.valueOf(loc.getLongitude())));
    } else {
        lblLatitud.setText("Latitud: (sin_datos)");
        lblLongitud.setText("Longitud: (sin_datos)");
        lblPrecision.setText("Precision: (sin_datos)");
    }
}
}

```

Para inicializar una actualización de la posición, se realiza un envío manual de una nueva posición al emulador de Android, para simular que se cambió la localización. En la pestaña Emulador Control del DDMS, en la sección Location Controls. Introducir las coordenadas de longitud y latitud y digitar Send, ello ejecutará el evento `onLocationChanged()` y se mostrarán estos datos en la interfaz, como se indica enseguida en las figuras 1 y 2:

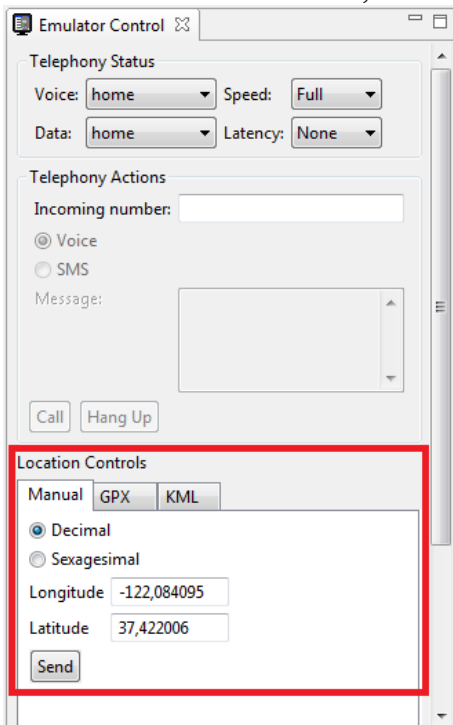


Figura 1. El cuadro Emulador Control.



Figura 2. Muestra latitud y longitud.

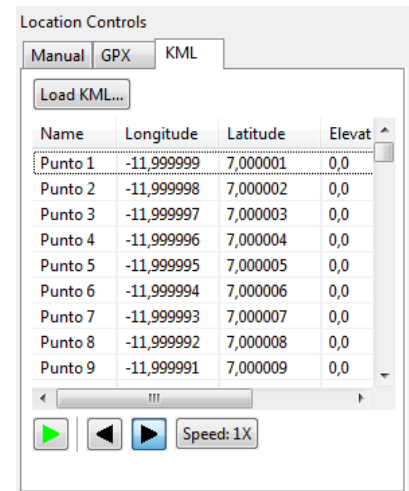


Figura 3. Uso del archivo KML.

También se puede proporcionar una lista de coordenadas con dos formas: en formato GPX o en un archivo KML. En este caso se genera un archivo KML con 1000 posiciones geográficas aleatorias.

Acceder a Location Controls y digitar en la pestaña GPX o KML. Por ejemplo, para utilizar un archivo KML se digita en el botón Load KML... , como se muestra en la figura 3, y ver la lista de coordenadas, las cuales se pueden navegar con los botones incluidos, como también se indica en la figura 3.

En el emulador, digitar Activar para detectar los cambios de posición y enseguida digitar en el botón verde. Observar que los valores de latitud y longitud se actualizan varias veces. Permitir que durante un minuto se ejecute la aplicación y después detener la captura de posiciones digitando Desactivar. Observar que en la ventana log del DDMS se muestran los mensajes de log. Se debe mostrar algo similar a lo siguiente:

```

05-08 10:50:37.921: INFO/LocAndroid(251): 7.0 - -11.999998333333334

```

05-08	10:50:38.041:	INFO/LocAndroid(251):	Provider Status: 2		
05-08	10:50:38.901:	INFO/LocAndroid(251):	7.0000016666666666	-	-11.9999966666666668
05-08	10:50:39.941:	INFO/LocAndroid(251):	7.0000016666666666	-	-11.9999966666666668
05-08	10:50:41.011:	INFO/LocAndroid(251):	7.0000033333333333	-	-11.9999950000000002
05-08	10:50:43.011:	INFO/LocAndroid(251):	7.0000050000000001	-	-11.9999933333333334
05-08	10:50:45.001:	INFO/LocAndroid(251):	7.0000066666666667	-	-11.9999916666666665
05-08	10:50:46.061:	INFO/LocAndroid(251):	7.0000083333333333	-	-11.9999899999999999
05-08	10:50:47.131:	INFO/LocAndroid(251):	7.0000083333333333	-	-11.9999899999999999
05-08	10:50:47.182:	INFO/LocAndroid(251):	Provider Status: 1		
05-08	10:51:02.232:	INFO/LocAndroid(251):	7.0000233333333333	-	-11.999975
05-08	10:51:02.812:	INFO/LocAndroid(251):	7.0000233333333333	-	-11.9999733333333333
05-08	10:51:02.872:	INFO/LocAndroid(251):	Provider Status: 2		
05-08	10:51:03.872:	INFO/LocAndroid(251):	7.0000249999999999	-	-11.9999733333333333
05-08	10:51:04.912:	INFO/LocAndroid(251):	7.0000266666666668	-	-11.9999716666666665

Observar que se muestran los siguientes detalles:

- Un grupo inicial de actualizaciones entre las 10:50:37 y las 10:50:47, con 8 lecturas.
- Otro grupo de actualizaciones entre las 10:51:02 y las 10:51:13, con 11 lecturas.
- Un grupo más de actualizaciones entre las 10:51:28 y las 10:51:38, con 10 lecturas.
- Entre cada grupo de lecturas se transcurre un tiempo aproximado de 15 segundos.
- Los grupos se forman con un número variable de lecturas.

Buscar en el mensaje log que:

- Después de recibir cada grupo de lecturas, el proveedor pasa a estado 1 (TEMPORARILY_UNAVAILABLE).
- Después de iniciar la recepción de nuevas lecturas, el proveedor pasa a estado 2 (AVAILABLE).

NOTA. El archivo KML de muestra se incluye en la carpeta adjunta.